

NetCon V2 Full Documentation



A Modular Networking and Lifecycle Management Framework for Roblox

<https://github.com/CTTeddy/NetConV2/>

Overview

NetCon V2 is a modular networking framework for Roblox that structures client-server communication and manages lifecycles. Unlike V1, it dynamically handles individual remotes and features Knit-inspired auto-loading.

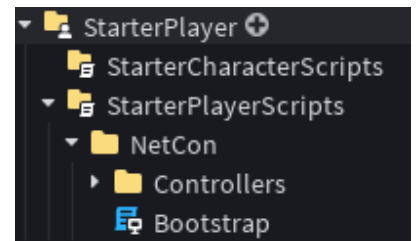
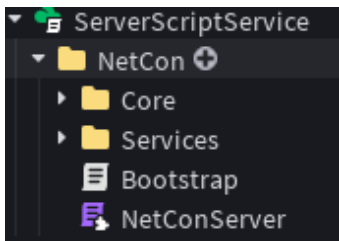
Feature	Knit	NetCon V1	NetCon V2
Focus	Full framework (networking + lifecycle).	Single-channel networking wrapper.	Modular networking & lifecycle framework.
Architecture	Dedicated signals & properties.	One global channel (actionName filter).	Dynamic individual remotes per event.
Scope	Large-scale, production-ready games.	Minimal, for small scripts/setups.	Balanced, structured, low-boilerplate.

Project Architecture

NetCon V2 organizes its code into dedicated folders across the workspace hierarchy:

- **ReplicatedStorage / NetCon**
 - Events/ (Auto-generated folder containing RemoteEvent instances)
 - Shared/ (Contains the client module loader and NetCon client API)

- **ServerScriptService / NetCon**
 - Core/ (Event, Network, Registry, and ServiceLoader modules)
 - Services/ (Server-side service modules loaded automatically)
 - Bootstrap – Starts The System Server Side
 - NetConServer (Main server initialization script)
- **StarterPlayer / StarterPlayerScripts / NetCon**
 - Controllers/ (Client-side controller modules loaded automatically)
 - Bootstrap – Starts The System Client Side



Server-Side API (NetConServer)

The server script initializes the core network system, loads all services, and manages remote communication.

Methods:

- **NetConServer.Init()**
 - Starts the network layer, loads and runs all server services via ServiceLoader, and logs the startup warning.

Example:

```
local NetConServer = require(script.Parent.NetConServer)
NetConServer.Init()
```

-
- **NetConServer.Event(name)**
 - Creates or retrieves a managed server event wrapper.
 - **Event Object Methods:**
 - :OnServer(callback): Registers the function executed when a client triggers the event.
 - :SetCooldown(seconds): Applies a rate-limit cooldown per player.
 - :SetValidator(callback): Validates incoming payloads; returns false to block execution.
 - :FireClient(player, data): Sends data to a specific player.
 - :FireAll(data): Broadcasts data to all connected clients.
 - **NetConServer.Function(name)**
 - Creates or retrieves a RemoteFunction instance.
 - **NetConServer.Register(name, callback)**
 - Quick helper to create an event and assign its server callback simultaneously.
 - **NetConServer.GetService(name)**

- Fetches a running service instance from the central registry.

Client-Side API (NetCon)

The client module handles communication targeting the server-side architecture.

Methods:

- **NetCon.Init()**
 - Initializes the client side and outputs the startup signature.
- **NetCon.Event(name) / NetCon.Fire(name, data)**
 - Safely references or fires an event payload to the server.

Example:

```
local NetCon = require(game.ReplicatedStorage.NetCon.Shared.NetCon)
NetCon.Fire("SampleEvent", { action = "test" })
```

- **NetCon.On(name, callback)**
 - Listens for triggers sent from the server.
- **NetCon.Function(name)**
 - References a remote function.
 - **Function Object Methods:**
 - **:Invoke(data):** Sends an invocation to the server and yields for a response.

Service & Controller Lifecycles

NetCon V2 automatically scans specific directories to handle execution flows.

Server Services (ServerScriptService > NetCon > Services)

Any ModuleScript inside this folder is automatically loaded. If it features a **:Start()** method, it runs inside an asynchronous protected call.

```
local MyService = {}

function MyService:Start()
    print("MyService started successfully.")
end

return MyService
```

Client Controllers (StarterPlayerScripts > NetCon > Controllers)

Any ModuleScript placed here is loaded when the client boots up, executing its :Start() method automatically via task.spawn.

```
local MyController = {}

function MyController:Start()
    print("MyController started successfully.")
end

return MyController
```



Strict Terms of Use & Copyright Notice

By accessing, downloading, integrating, or utilizing NetCon v2, you agree to the following legally binding terms:

All Rights Reserved: Exclusive intellectual property of the developer.

Absolute Prohibition of Modification: Altering any portion of the core source code is strictly prohibited.

The startup logs and copyright warnings containing **NetCon V2 Server/Client By CTTeddy** cannot be removed or altered under any circumstances.

Strict Anti-Redistribution & Anti-Monetization Policy: Redistribution, resale, or unauthorized distribution is forbidden.

Enforcement & Legal Action: Unauthorized use will result in DMCA takedowns and formal legal action.



Credits & Metadata

Developer: CTTeddy

Project: NetCon v2 Simple Network & Communication System